xAI Group Week 4 Summary

Talk presented by Kaustuva, summary by Aritra

based on "Are Your Explanations Reliable?" Investigating the Stability of LIME in Explaining Text Classifiers by Marrying XAI and Adversarial Attack

Text Classification and LIME

1. Example of "Movie was visually stunning but boring and predictable"
2. Perturb the sentence and see changes in prediction by the black box model. By perturb, we mean remove words (one or many) or replace them with similar words or phrases.
3. LIME will fit a sparse linear model locally according to the predictions by the black box model and provide an explanation.
4. The LIME explanation pipeline was briefly shown.

Stability of LIME

1. What is stability in the context of text classification and LIME explanations? "Small changes in input should produce small changes in explanations."
2. Is LIME itself unstable or is it due to the parameters and input?
3. Rakhi Ma'am pitched that LIME may be unstable due to the sampling it does, when we look at the scenario procedurally.
4. KEY QUESTION: Does LIME produce different and inconsistent explanations even when the model and input remain the same?

LIME Explanation Generation

To explain prediction f(d), LIME does the following steps:

Step 1: Generate perturbations

- Randomly select k words from document d (the document is a singular instance).
- Remove all occurrences of those words.
- This produces a perturbed document d_i. (multiple instance of d_i)

Step 2: Sampling

Repeat Step 1 to generate $\{d_i\}_{i=1}^{n}$ where *n* is the sampling rate.

Step 3: Train surrogate model

Train an interpretable model g(.) using features d_i and labels $f(d_i)$

<u>Sampling Rate Importance</u>:
The parameter n determines the number of local samples used to train the surrogate model.

<u>Small Sampling Rate (n)</u>

- Insufficient perturbation samples
- Poor surrogate model g (·)
- Unstable explanations

<u>Large Sampling Rate (n)</u> (A larger n gives a better result, intuitively and empirically)

- Better approximation of the target model
- More reliable explanations
- However increases computational cost

As ma'am had said earlier, randomness in sampling can lead to different explanations for same output.

Some numerical evidence from the paper for the same: When changing from default n=5000, average similarity decreases by approximately 16% to 28.5%, with small sampling rates having highly different explanations.

Some clarification and doubts in this portion:

- For better understanding, consider the example of the movie sentence presented at the start. Here the features will be "boring", "stunning", "visually", "predictable". They are removed or replaced by say "boring" with "slow" or "predictable" with "formulaic".

- There was slight confusion between k and n. k words are chosen to be removed for a single perturbation. Every perturbation has some changes, say k words are removed or replaced. The number of perturbations is n.

Text Adversarial Optimization Problem

We try to frame our goal as an objective function. That is, for original document d_b and perturbed document d_p, let e_b and e_p be their explanations generated by LIME. We want to find a perturbed document that minimizes the similarity between explanations, i.e.,

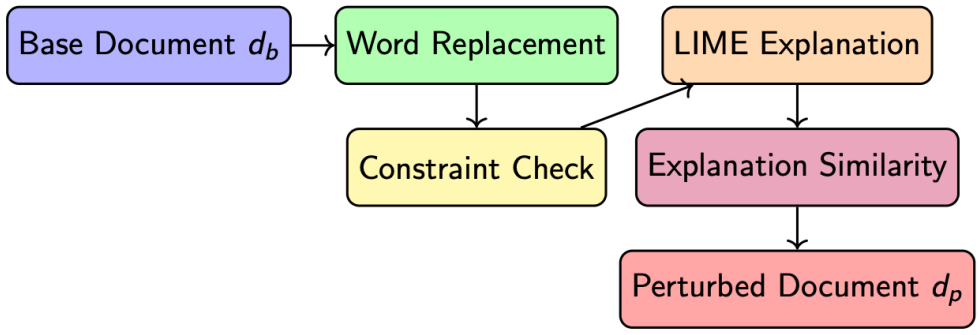
$$d_p = argmin_{d_p} Sim_e \left(e_b , e_p\right)$$

The prediction must be the same but the explanation must be different.

Constraints:

1. **Prediction Constraint**: $f(d_b) = f(d_p)$ (Same prediction)
2. **Semantic Similarity Constraint**: $Sim_s(d_b, d_p) \geq \delta$ (The perturbed document must be semantically similar to the original by a large enough threshold)
3. $i \leq \epsilon \cdot |f|$ (**Limiting number of perturbations** so that meaning is not altered too much). Here i is number of performed perturbations, |f| is total number of unigram bag-of-words features and ϵ is a hyperparameter controlling the perturbation budget.
4. **Feature Overlap Constraint**: $e_p \cap c \neq \emptyset \; \forall \, c \in e_b^{[:k]}$, i.e., The top k important features in original explanation e_b should be in e_p, so that we can make sure meaning is unaltered as important features still remain. The explanations are generated using LIME.

XAIFOOLER Attack Pipeline



Objective function: $\arg\min Sim_e(e_{d_b}, e_{d_p})$

subject to

$$f(d_b) = f(d_p), \quad Sim_s(d_b, d_p) \geq \delta, \quad i \leq \epsilon \cdot |f|, \quad e_{d_p} \cap c \neq \emptyset \quad \forall \, c \in e_{d_b}^{[:k]}$$

XAIFOOLER, a greedy algorithm designed to manipulate explanations produced by text-based XAI methods.

- XAIFooler is **not improving LIME**. **It is testing the stability of LIME**. If through XAIFooler, we get a perturbed document with a sufficiently different explanation than the original document, it will prove the instability of LIME.
- Explanations are represented as a ranked list of words ordered by their importance
- While perturbing, we focus on keeping the **prediction** and **semantic meaning of the document** the same, while trying to get **semantically different explanations**.

<u>Step 1</u>

We remove stop words and the first k important keywords first and then consider the remaining words in original d_b. For each word, we remove it individually and observe the change in the model prediction and order the words in descending order according to the change in prediction that their removal brought. We want to modify words with lower predictive importance and change the explanations through them, to showcase instability while keeping semantic similarity.

<u>Step 2</u>

Replace each word with its candidate substitutions. Accept replacement only if explanation similarity reduces while the constraints are satisfied. Where do we get the replacements from? The Paraphrase Database (PPDB) is an extensive semantic resource, consisting of a list of phrase pairs with (heuristic) confidence estimates. Candidate replacements must maintain the same part-of-speech (Noun remains noun and verb remains verb) and remain within a similarity threshold.

XAIFOOLER Algorithm

Algorithm 1 Adversarial Explanation Generation

1: *Input:* **target model** f, **Original Document** d_o, **Base Explanation** e_b, **Maximum Perturbation Threshold** p_t, **Current Perturbations** p_c **Current Similarity** s
2: *Output:* **Perturbed Document** d_p, **Perturbed Explanation** e_p, **Updated Similarity** s
3: *Initialize:* $e_p \leftarrow e_b$, $i \leftarrow 0$, $s \leftarrow 1$, $p_c \leftarrow 0$
4: *Initialize:* I $\leftarrow$ indices of replacement candidates that satisfy C
5: **while** $I \neq \varnothing$ *and* $p_c < p_t$ **do**
6: $\quad s_i$ = RBO(d_p, perturb($d_p[I[i]]$)
7: $\quad$ **if** $s_i < s$ **then**
8: $\quad\quad s \leftarrow s_i$
9: $\quad\quad e_p \leftarrow$perturb($d_p[I[i]]$)
10: $\quad$ **end if**
11: $\quad I[i] = \varnothing$
12: $\quad i \leftarrow i + 1$
13: **end while**
14: **return** (d_p, e_p, s)

Discussed points here:

- C comes from PPDB, and it is paired, that is for our movie example, “boring” and “slow” are chained together. “slow” cannot replace “stunning”, which carries a different semantic meaning in the document.
- RBO is performing semantic similarity on the **explanations** of the perturbed document and its new perturbation, not the documents itself.
- The d_P is not explicitly updated, only the explanations are. Kaustuva da mentioned if he wrote this algorithm, he would compare against a placeholder.
- RBO is performed on a perturbed document and then a new perturbation of that, and the original document was only considered in the first iteration. There was considerable discussion on this.
- Step 4 in the algorithm has the most computational cost
- Thanks to Sagnik, there was considerable clarification on the term perturb(d_p[I[i]]). Here, we are perturbing d_p by changing the i-th word in the list of candidates I. That is, the while loop is iterating through the list of candidates I, and we are calculating the semantic similarity of the explanation of new perturbation peturb(d_p[I[i]]) with the previous d_p (according to our interpretation of the algorithm). We get this new perturbation by at every iteration, changing the i-th word in our list of candidates I.

- We try to find the minimum similarity through our list I, and we want this to below the threshold of s (initialised to 1) in the algorithm.
- There was a short discussion on the number of perturbed documents we are able to generate through this algorithm. Rakhi ma'am surmised we would get at least one, the original document and probably many more. Kaustuva da noted that even if we get one different document from the original, we have proved the instability of LIME.
- I am still confused why we are not comparing with d_b (d_0 in the algorithm), and instead we are comparing with the previous perturbation, is it because we are greedy? Rakhi Ma'am asked if there was some sort of monotonicity between d_b, d_p and the new perturbation on d_p.
- Ritika mentioned we can still calculate similarity between d_b and d_p and why are we not?
- Rakhi Ma'am mentioned that she would do it with a for loop by changing one word at a time, which would not be greedy and more computationally expensive I guess.

Rank Biased Overlap (RBO)

Measures like Jaccard's (compares two sets by computing the ratio of the shared elements between the sets to the union of both sets) and Kendall's τ & Spearman's ρ (ignores weights, the capacity to handle unequal list lengths, and disjoint features) have their own problems. The paper uses the novel Rank Biased Overlap measure.

$$RBO = (1-p)\sum_{d=1}^{\infty} p^{d-1} A_d$$

$$A_d = \frac{|L_1^d \cap L_2^d|}{d}$$

For RBO, we take two lists, their top d features and calculate the cardinality of how many features are common between these two sets.

It has weighted similarity across ranking depth and top features contribute more. Further, for parameter p, if it is closer to zero, it considers the top ranked features. If it is approaching 1, then it takes the deeper features, as E[D]=1/(1-p), for depth D.

Conclusion and Future

To conclude, LIME explanations are vulnerable to adversarial manipulation.

Limitations:

- XAIFOOLER only evaluates LIME and not other xAI like SHAP.
- Greedy algorithm may not give optimal perturbation.
- The computational cost of running this is high due to the large number of perturbations and their explanations.
- Also, all of this depends on the surrogate model LIME chooses to do the explanation, so not totally model agnostic.

Future Directions:

- What is the real source of instability within LIME?
- Stability analysis of SHAP explanations
- Theoretical bounds (What parameters need to be tuned to make explanations robust)
- Defense mechanisms against explanation attacks
- Robust explanation frameworks

Parting Discussion

- Any numerical data? Tough to show with the huge volume of any single document on which XAIFooler is tested on to show the instability.
- This is on top of LIME, not as a feature of LIME but instead stress-testing LIME of sorts. This gets confusing because LIME is also perturbing and jittering data locally to get explanations and then we are perturbing on top of it. But the key difference is that we are perturbing to get different explanations and that is our target, whereas in LIME, we perturb to get the explanation itself.
- We are targeting the importance of features that LIME attributes and how perturbations are changing them to create a different explanation, bringing the vast instability. (for example, peptic ulcer being the prediction but the explanation changing from joint pain and winter to rash and worse).